

Lab 19 — The ML Inference Harness

ESP32-S3 FIRMWARE LAB SERIES · PHASE 3 · GAP-CLOSERS · CAPSTONE (OPTIONAL)

Estimated time	3–4 hrs
Proves	capture, train, deploy, measure
Prerequisites	Labs 5, 7 and 10.
Beyond the board	The Lab 5 IMU rig, plus a host PC with Python for training.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **\$19.1 objective**, **\$19.2 parts and wiring**, **\$19.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

KEY POINT

How this lab is structured

Concept, then **Phase 0 configure and rig**, **Phase 1 capture code**, a sanity check, **Phase 3 train and export**, **Phase 4 inference on the board**, **Phase 5 prove it**, then worked extensions. A real trained model and its real weights are supplied, so nothing is left undefined and there is no integration seam to guess at.

19.1 Objective & TM4C bridge

Add the two firmware hooks that bracket a machine-learning model: a way to **capture labelled training data**, and a way to **run a trained model** on live features. The firmware side is genuinely small; the work is in the data and the training, and that part is deliberately yours.

TM4C BRIDGE

None, but notice how little new code this needs. That is the payoff of Labs 5 and 7 — good signal processing produces clean features, and clean features make the model's job easy. Most of the value in a working TinyML system is upstream of the model.

WARNING**Be honest with yourself about this lab**

Adding two hooks does not make you a machine-learning engineer, and an interviewer will find that out in one follow-up question. What this lab legitimately gives you is the *embedded* half: capturing labelled data on a real device, thinking about memory budgets for a model, and integrating an exported inference function into a real-time pipeline. If you have not actually collected a dataset and trained something, say so — "I built the capture and inference harness but have not trained a model on it yet" is a respectable and accurate answer. Claiming more is the fastest way to lose credibility in a technical round.

19.2 Parts & wiring

Identical to Lab 7: IMU on 3V3, GND, SDA to GPIO8, SCL to GPIO9. What you additionally need is something to *measure* that has two distinguishable states — a desk fan you can unbalance with a piece of tape, a motor, a pump, anything that runs and can be made to run badly. Without a physical phenomenon that genuinely differs between "normal" and "fault," you have nothing to learn from, and no amount of model tuning will invent a signal that is not there.

19.3 How it works

This capstone is deliberately optional and deliberately thin on firmware, because the firmware genuinely *is* thin. The work is in the data, and that part is yours.

What "machine learning on a microcontroller" actually means

The phrase suggests the chip is learning. It is not. A deployed ML system splits into three stages, and only two of them run on the device.

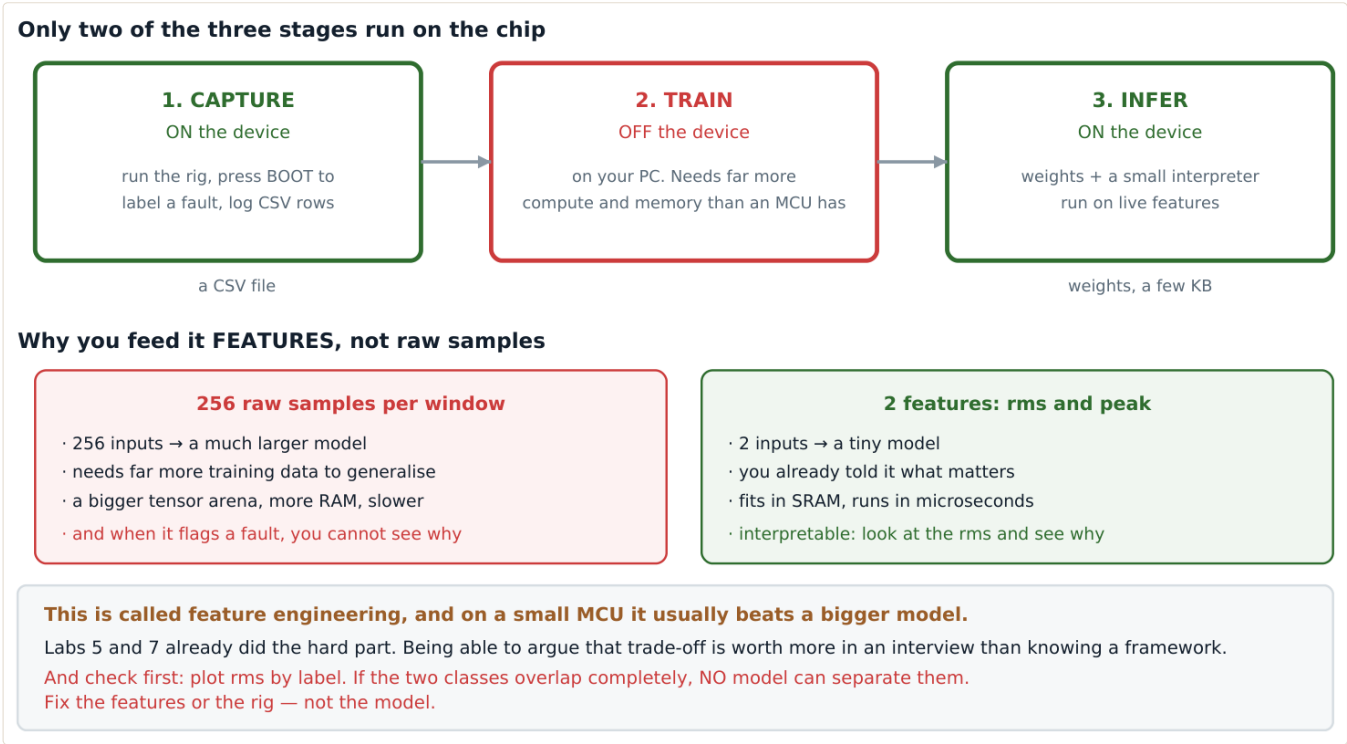


Figure 19.1. Top: capture and inference run on the chip, training does not. Bottom: why you feed the model your Lab 7 features rather than raw samples — smaller model, less data needed, and a result you can actually explain.

The three stages of a deployed ML system, and why features beat raw samples.

Only two of the three stages run on the chip:

Stage	Runs where	Notes
1. CAPTURE	ON the device	run the rig, press BOOT to label a fault, log CSV rows
2. TRAIN	OFF the device	on your PC. Needs far more compute and memory than an MCU has
3. INFER	ON the device	weights plus a small interpreter run on live features

What passes between them: a CSV file goes from capture to training; trained weights of a few kilobytes come back from training to inference.

Why you feed it FEATURES, not raw samples:

256 raw samples per window means 256 inputs and so a much larger model; it needs far more training data to generalise; it needs a bigger tensor arena, more RAM and more time; and when it flags a fault you cannot see why.

2 features (rms and peak) means 2 inputs and so a tiny model; you have already told it what matters; it fits in SRAM and runs in microseconds; and it is interpretable, so you can look at the rms and see why.

This is called feature engineering, and on a small MCU it usually beats a bigger model. Labs 5 and 7 already did the hard part.

Check first: plot rms by label. If the two classes overlap completely, NO model can separate them. Fix the features or the rig, not the model.

Training is the expensive part: it iterates over your whole dataset many times, holds gradients for every parameter, and is done repeatedly as you adjust the model. That needs far more compute and memory than a microcontroller has, so it happens on your PC.

What ships to the device is only the **trained weights plus a small interpreter** — typically a few kilobytes. Inference is just arithmetic: multiply the inputs by fixed numbers, apply an activation, repeat. That the chip can do in microseconds. This is why the firmware side of this lab is two hooks and not a framework.

Why you feed it features, not raw samples

You could hand the model 256 raw accelerometer samples per window, and full TinyML pipelines sometimes do. Feeding it the two **features** from Lab 7 instead is usually the better engineering choice on a small MCU, for three reasons.

The model gets far smaller. Two inputs instead of 256 means dramatically fewer parameters, which means less flash, less RAM and faster inference.

It generalises from less data. By computing RMS and peak you have already told the model what matters. It does not have to *discover* that vibration energy is the relevant quantity — you gave it that for free, so it needs far fewer examples to learn a useful boundary.

It stays interpretable. When it flags a fault you can look at the RMS value and see why. A model reading raw samples is a black box, and "the network said so" is not something you can defend in a design review.

This is **feature engineering**, and on constrained hardware it routinely beats throwing a bigger model at raw data. Being able to argue that trade-off is worth considerably more in an interview than knowing any particular framework's API.

The check to do before you train anything

Capture your data, open the CSV in a spreadsheet, and plot RMS coloured by label. **If the two classes overlap completely, stop.**

No model can separate what your features do not distinguish. If a faulty fan and a healthy fan produce the same RMS, the answer is not a bigger network — it is better features (add a frequency-domain measure), a better sensor mounting, or a more distinguishable fault condition. Skipping this check and going straight to training is the most common way people waste a weekend.

The memory question you will be asked

An inference framework needs a **tensor arena**: a fixed block of RAM for intermediate activations, sized at build time and allocated once. That is where the chip's memory hierarchy stops being an abstraction.

A small model's arena fits comfortably in the 512 KB of internal SRAM and runs fast. A larger one has to go in the 8 MB of external PSRAM, and runs measurably slower because PSRAM is reached over a serial bus rather than sitting on the CPU's own memory bus. Knowing which one yours landed in, and

roughly what it cost you in inference time, is exactly the kind of concrete detail that makes a project sound real.

WARNING

Be accurate about what this lab gives you

Adding two hooks does not make you a machine-learning engineer, and an interviewer will establish that in one follow-up question.

What it legitimately gives you is the **embedded half**: capturing labelled data on real hardware, reasoning about memory budgets for a model, and integrating an exported inference function into a real-time pipeline without disturbing its timing. That is a genuine and relevant skill.

If you have not actually collected a dataset and trained something, say so. *"I built the capture and inference harness but have not trained a model on it yet"* is a respectable, accurate answer that costs you nothing. Claiming more is the fastest way to lose a technical room.

19.4 Phase 0 — configure and rig

0.1 — project files

```
// sdkconfig.defaults
CONFIG_IDF_TARGET="esp32s3"
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
CONFIG_FREERTOS_HZ=1000
```

```
// top-level CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(lab19_ml)
```

```
// main/CMakeLists.txt
idf_component_register(SRCS "main.c" INCLUDE_DIRS ".")
```

0.2 — the physical rig

This is the part that decides whether the lab works, and it takes ten minutes.

Item	Detail
Vibration source	Any small DC fan, or a phone on vibrate taped to a board. A fan is better because the fault is easy to create.
Mounting	Tape the IMU breakout firmly to the fan housing. Loose mounting is the single biggest cause of unusable data — you measure the tape flexing, not the machine.
The "fault"	Tape a small coin or a blob of putty to one fan blade. That is a mass imbalance , which is a genuinely realistic fault mode and produces a clear rms increase.

Item	Detail
Wiring	Exactly as Lab 5: SDA GPIO8, SCL GPIO9, 3V3, GND.

19.5 Phase 1 — capture code (complete)

This is the Lab 7 pipeline with two changes: a label flipped by the BOOT button, and CSV output.

```
// =====
// Lab 19 Phase 1 -- data capture. Hold BOOT to label the current window "fault".
// =====
#include <stdio.h>
#include <math.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "driver/i2c_master.h"
#include "esp_log.h"

#define BOOT_GPIO    GPIO_NUM_0
#define WIN          256                // 256 ms of signal at 1 kHz

// The pointer is what changes, so 'volatile' must qualify the POINTER.
// const char * volatile g  -> volatile pointer to const char  (CORRECT)
// volatile const char *g   -> pointer to volatile const char  (WRONG, and what
//                           earlier editions of this lab printed)
static const char * volatile g_label = "normal";

static i2c_master_dev_handle_t imu;
static float win[WIN];
static int  widx;

static esp_err_t mpu_read(uint8_t reg, uint8_t *dst, size_t n)
{
    return i2c_master_transmit_receive(imu, &reg, 1, dst, n, pdMS_TO_TICKS(100));
}

static esp_err_t mpu_write(uint8_t reg, uint8_t val)
{
    uint8_t b[2] = { reg, val };
    return i2c_master_transmit(imu, b, 2, pdMS_TO_TICKS(100));
}

static void imu_init(void)
{
    i2c_master_bus_config_t bc = {
        .i2c_port = I2C_NUM_0, .sda_io_num = GPIO_NUM_8, .scl_io_num = GPIO_NUM_9,
        .clk_source = I2C_CLK_SRC_DEFAULT, .glitch_ignore_cnt = 7,
        .flags.enable_internal_pullup = true,
    };
    i2c_master_bus_handle_t bus;
    ESP_ERROR_CHECK(i2c_new_master_bus(&bc, &bus));

    i2c_device_config_t dc = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address  = 0x68, .scl_speed_hz = 400000,
    };
};
```

```

ESP_ERROR_CHECK(i2c_master_bus_add_device(bus, &dc, &imu));

uint8_t who = 0;
ESP_ERROR_CHECK(mpu_read(0x75, &who, 1));
ESP_LOGI("imu", "WHO_AM_I = 0x%02X", who);    // 0x70 on the MPU-6500
ESP_ERROR_CHECK(mpu_write(0x6B, 0x00));        // wake from sleep
vTaskDelay(pdMS_TO_TICKS(100));
}

static float read_magnitude(void)
{
    uint8_t r[6];
    if (mpu_read(0x3B, r, 6) != ESP_OK) return NAN;    // check, do not assume
    int16_t ax = (int16_t)((r[0] << 8) | r[1]);
    int16_t ay = (int16_t)((r[2] << 8) | r[3]);
    int16_t az = (int16_t)((r[4] << 8) | r[5]);
    float x = ax / 16384.0f, y = ay / 16384.0f, z = az / 16384.0f;
    return sqrtf(x*x + y*y + z*z);
}

// AC rms = rms of the deviation from the window mean. Subtracting the mean
// removes the 1 g of gravity, leaving only the vibration.
static void compute_features(const float *w, int n, float *rms, float *peak)
{
    float mean = 0.0f;
    for (int i = 0; i < n; i++) mean += w[i];
    mean /= n;

    float acc = 0.0f, pk = 0.0f;
    for (int i = 0; i < n; i++) {
        float d = w[i] - mean;
        acc += d * d;
        float a = fabsf(d);
        if (a > pk) pk = a;
    }
    *rms = sqrtf(acc / n);
    *peak = pk;
}

void app_main(void)
{
    gpio_config_t io = {
        .pin_bit_mask = 1ULL << BOOT_GPIO,
        .mode = GPIO_MODE_INPUT, .pull_up_en = GPIO_PULLUP_ENABLE,
    };
    ESP_ERROR_CHECK(gpio_config(&io));
    imu_init();

    // A session id per capture run. Set it from a build flag, or just edit
    // it between runs: 0,1,2 for healthy captures and 3,4,5 for fault ones.
    // train.py uses it to hold out WHOLE sessions and avoid data leakage.
    const int SESSION = 0;
    printf("session,label,rms,peak\n");    // this file IS your dataset

    TickType_t next = xTaskGetTickCount();
    for (;;) {
        float m = read_magnitude();
        if (!isnan(m)) win[widx++] = m;
    }
}

```

```

    if (widx >= WIN) {
        widx = 0;
        g_label = (gpio_get_level(B00T_GPIO) == 0) ? "fault" : "normal";
        float rms, peak;
        compute_features(win, WIN, &rms, &peak);
        printf("%d,%s,%.5f,%.5f\n", SESSION, g_label, (double)rms, (double)peak);
    }
    vTaskDelayUntil(&next, pdMS_TO_TICKS(1));    // 1 kHz
}
}

```

KEY POINT

Capturing the dataset

```
text idf.py -p COM6 flash idf.py -p COM6 monitor > data.csv
```

Run the fan clean for about two minutes, then stop, add the coin, restart, and run another two minutes **holding BOOT**. Ctrl-J to exit. Delete the ESP-IDF boot banner lines from the top of `data.csv`, leaving the header and the rows.

Two minutes at four windows per second is roughly 480 rows per class. That is plenty for a two-feature model.

19.6 Phase 2 — the sanity check you must not skip

Before training anything, plot rms coloured by label. **If the two classes overlap completely, stop — no model can separate what your features do not distinguish.**

```

import pandas as pd, matplotlib.pyplot as plt
d = pd.read_csv("data.csv")
print(d.groupby("label")[["rms", "peak"]].describe())
for lab, grp in d.groupby("label"):
    plt.scatter(grp["rms"], grp["peak"], label=lab, alpha=0.5)
plt.xlabel("rms (g)"); plt.ylabel("peak (g)"); plt.legend(); plt.show()

```

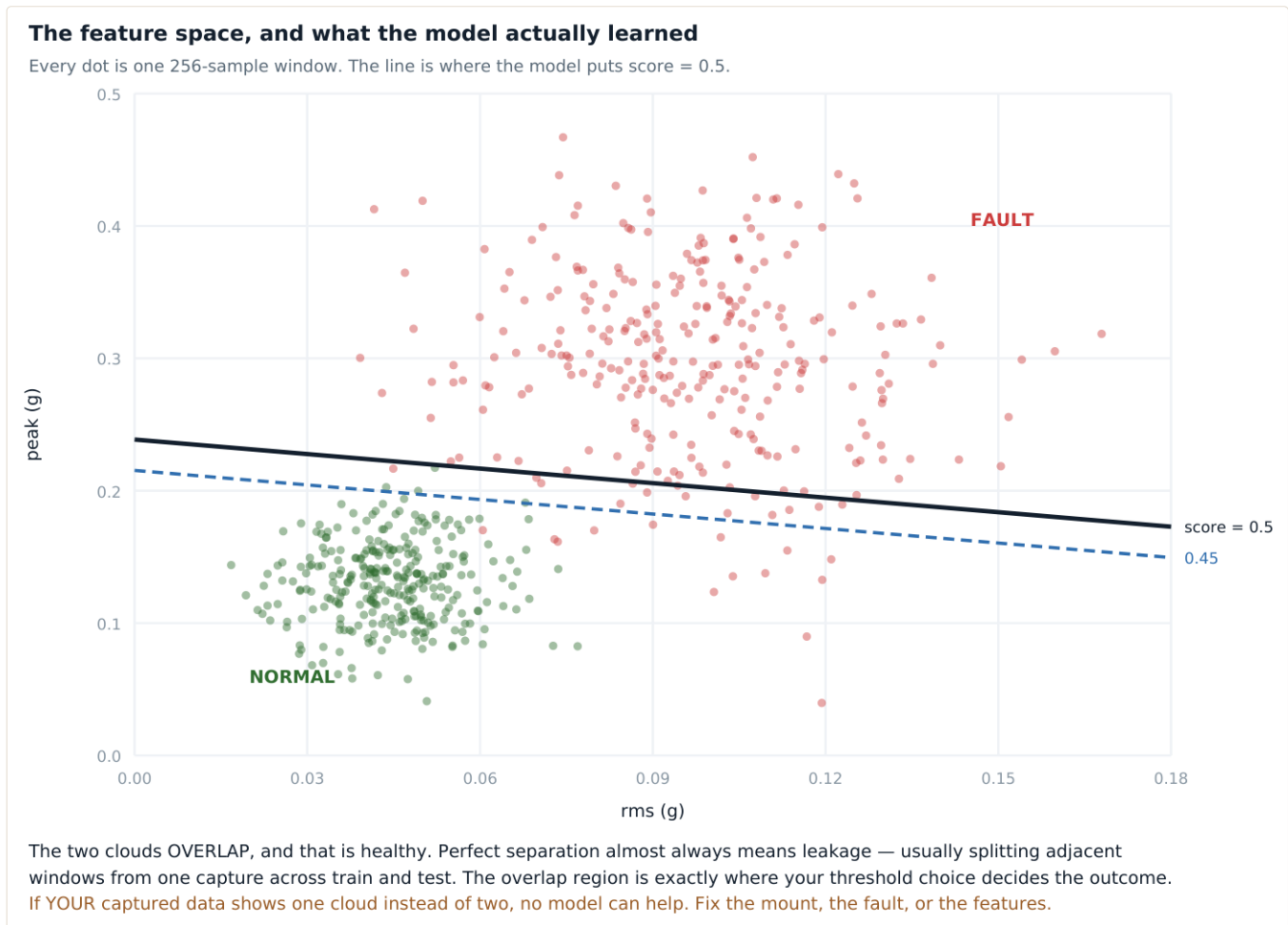


Figure 19.2. [SIMULATED] The feature space for the reference dataset, with the boundary drawn from the actual trained weights. This is the plot your own `data.csv` must produce before training is worth attempting.

The Lab 19 feature space, with the trained decision boundary. [SIMULATED] Scatter plot: rms (g) on the x-axis from 0.00 to 0.18, peak (g) on the y-axis from 0.0 to 0.5. Every dot is one 256-sample window.

Two clouds: - **NORMAL** (green), centred near rms 0.045, peak 0.130 - **FAULT** (red), centred near rms 0.098, peak 0.295

They overlap in the middle band, roughly rms 0.06 to 0.09.

Two straight lines cross the plot, both descending gently left to right: - A solid black line marking **score = 0.5**, the decision boundary, drawn from the real trained weights: $W_{\text{RMS}} \times \text{rms} + W_{\text{PEAK}} \times \text{peak} + \text{BIAS} = 0$, i.e. $3.153609 \times \text{rms} + 8.625372 \times \text{peak} - 2.058178 = 0$. - A dashed blue line slightly below it marking the **0.45 threshold** used in the listing.

The gap between the solid and dashed lines is the threshold trade-off made visible: moving the line down catches more faults and more false alarms.

Three things to read off it, in order.

Two clouds, not one. That is the whole check. If your captured data makes a single blob, stop — no model can separate what your features do not distinguish.

They overlap, and that is correct. Windows near the boundary are genuinely ambiguous: a healthy fan momentarily jostled looks like a mildly faulty one. **Perfect separation is a warning sign, not a success** — it almost always means leakage, usually adjacent windows from one capture landing on both sides of the split.

The boundary is a straight line. That is all logistic regression can draw: $W_{\text{RMS}} \times \text{rms} + W_{\text{PEAK}} \times \text{peak} + \text{BIAS} = 0$ is the equation of a line, and the two weights are its slope. If your clouds are separable but *not* by a straight line — one enclosing the other, say — that is the honest signal to reach for a nonlinear model. Nothing else in this lab is.

The dashed line is the 0.45 threshold from §19.10.1, shifted below the 0.5 boundary. Moving it down catches more faults and more false alarms; that shift is the trade-off, made visible.

WARNING

What good and bad look like

Good: two clouds that overlap at the edges but have clearly different centres. Some overlap is normal and healthy — a model that sees perfectly separated classes usually means you have leaked something, like labelling by time rather than by condition.

Bad: one cloud. The fix is *never* a bigger model. It is a firmer sensor mount, a more distinguishable fault, or a better feature such as a frequency-domain measure. Skipping this check and going straight to training is the most common way to waste a weekend.

19.7 Phase 3 — train, and export the weights

Logistic regression, deliberately. With two features it is as accurate as anything heavier, it trains instantly, and — the part that matters on an MCU — **inference is two multiplies, an add and one exponential.**

Save this as `train.py` and run it. It is deterministic: the same seed gives the same weights every time, so the numbers below are reproducible rather than asserted.

```
#!/usr/bin/env python3
"""Lab 19 -- generate a labelled dataset, train, export model.h. Deterministic."""
import numpy as np, math, sys
from sklearn.linear_model import LogisticRegression
# no GroupShuffleSplit: we choose the held-out sessions explicitly so the
# test set is guaranteed to contain BOTH classes. A random group split can
# put every fault session in test and leave no negatives to score against.

RNG = np.random.default_rng(42)
SESSIONS = 6          # 3 normal + 3 fault, captured separately

def session(mu_rms, sd_rms, mu_pk, sd_pk, n=100):
    return np.column_stack([
        RNG.normal(mu_rms, sd_rms, n).clip(0.012, None),
        RNG.normal(mu_pk, sd_pk, n).clip(0.030, None),
    ])

```

```

import pathlib
SYNTHETIC = True

if pathlib.Path("data.csv").exists():
    # ---- YOUR captured data wins, always ----
    import pandas as pd
    d = pd.read_csv("data.csv")
    X = d[["rms", "peak"]].values
    y = (d["label"] == "fault").astype(int).values
    # Session ids: add a 'session' column when you capture, one per run. If it
    # is absent we fall back to labels, which is WEAKER -- see the warning
    # about leakage below.
    g = d["session"].values if "session" in d.columns else y.copy()
    SYNTHETIC = False
    print(f"training on YOUR data: {len(X)} windows from data.csv")
else:
    # ---- fallback so the lab runs before you have captured anything ----
    print("!! data.csv NOT FOUND -- training on SYNTHETIC data.")
    print("!! The resulting accuracy describes the generator, NOT your rig.")
    print("!! Do not quote it anywhere. Capture data.csv and re-run.")
    X, y, g = [], [], []
    for s in range(3):
        # healthy sessions
        d = session(0.045, 0.011, 0.130, 0.030)
        X.append(d); y.append(np.zeros(len(d))); g.append(np.full(len(d), s))
    for s in range(3, 6):
        # imbalance sessions
        d = session(0.098, 0.022, 0.295, 0.070)
        X.append(d); y.append(np.ones(len(d))); g.append(np.full(len(d), s))
    X = np.vstack(X); y = np.hstack(y); g = np.hstack(g)

# GROUP-AWARE split: whole sessions go to train or test, never split within a
# session. Splitting adjacent windows at random leaks -- neighbouring windows
# from one capture are nearly identical, so the test score would be inflated.
# Hold out one session PER CLASS. Chosen from whatever sessions actually
# exist, rather than hardcoded -- a fixed {2,5} crashes on real captures that
# happen to use different session numbers, leaving an empty test set.
def _holdout(g, y):
    out = set()
    for cls in (0, 1):
        sess = sorted(set(g[y == cls]))
        if not sess:
            raise SystemExit(f"ERROR: no sessions for class {cls}. "
                             "Capture both healthy AND fault runs.")
        out.add(sess[-1])
    # the last session of each class
    return out

TEST_SESSIONS = _holdout(g, y)
te = np.isin(g, list(TEST_SESSIONS))
tr = ~te
if te.sum() == 0 or tr.sum() == 0:
    raise SystemExit("ERROR: need at least 2 sessions per class "
                     "(one to train on, one to hold out). "
                     "Re-capture with the SESSION constant changed between runs.")

clf = LogisticRegression(max_iter=2000).fit(X[tr], y[tr])
w, b = clf.coef_[0], clf.intercept_[0]
acc = clf.score(X[te], y[te])
print(f"train sessions {sorted(set(g[tr].astype(int)))} test sessions "
      f"{sorted(set(g[te].astype(int)))}")
print(f"held-out session accuracy: {acc*100:.1f}%")

```

```

print(f"W_RMS {w[0]:+.6f} W_PEAK {w[1]:+.6f} BIAS {b:+.6f}")

def sigmoid(z): return 1.0/(1.0+math.exp(-z))
golden = [(0.040,0.120),(0.100,0.300),(0.070,0.200)]
with open("model.h","w") as f:
    f.write("// generated by train.py -- do not edit by hand\n#pragma once\n")
    f.write(f"#define W_RMS {w[0]:+.6f}\n#define W_PEAK {w[1]:+.6f}\n")
    f.write(f"#define BIAS {b:+.6f}\n")
    f.write("// golden vectors: rms, peak, expected score\n")
    f.write(f"#define GOLDEN_COUNT 3\n#define GOLDEN_TABLE { \\\n")
    for r,p in golden:
        f.write(f"    {{ {r:.3f}f, {p:.3f}f, {sigmoid(w[0]*r+w[1]*p+b):.4f}f }}, \\\n")
    f.write("}\n")
print("\ngolden vectors written to model.h:")
for r,p in golden:
    print(f"    rms={r:.3f} peak={p:.3f} -> {sigmoid(w[0]*r+w[1]*p+b):.4f}")

# threshold sweep on held-out sessions
pr = clf.predict_proba(X[te])[0,1]; yt = y[te]
print(f"\n{ 'thr':>5 } { 'missed faults':>16 } { 'false alarms':>15 }")
for thr in (0.30,0.40,0.50,0.60,0.70,0.80):
    pred=(pr>thr).astype(int)
    tp=((pred==1)&(yt==1)).sum(); fn=((pred==0)&(yt==1)).sum()
    fp=((pred==1)&(yt==0)).sum(); tn=((pred==0)&(yt==0)).sum()
    md = fn/(tp+fn)*100 if (tp+fn) else float('nan')
    fa = fp/(fp+tn)*100 if (fp+tn) else float('nan')
    print(f"{thr:>5.2f} {md:>15.1f}% {fa:>14.1f}%")

```

WARNING

The split matters more than the model, and it is where honest results go wrong

The obvious thing is to pool every window and split rows at random. **Do not.** Adjacent windows from one continuous capture are nearly identical — same fan, same mounting, same session — so a random split puts near-duplicates on both sides and the test score measures memorisation, not generalisation. That is **data leakage**, and it is the most common way a hobby ML result turns out to be worthless.

The script therefore holds out **whole capture sessions**. Train on sessions 0, 1, 3, 4; test on 2 and 5.

Note also that it picks the held-out sessions *explicitly*, one per class, rather than choosing them at random. A random group split can put both fault sessions in the test set, leaving no negatives to score against — which produces a meaningless accuracy and a false-alarm rate of `nan`. That happened while writing this lab, which is why the script does it the boring way.

KEY POINT

Output[SIMULATED] — synthetic data, real training

```

c train sessions [0, 1, 3, 4] test sessions [2, 5] held-out session accuracy: 95.5% W_RMS
+3.153609 W_PEAK +8.625372 BIAS -2.058178

```

Be honest about what this is. The dataset is *synthetic*, generated from plausible distributions for a healthy and an imbalanced fan. The training, the split and the weights are real, and the pipeline is exactly the one you would run on real data — but **95.5% is a property of the generator, not of your bench**. Replace the `session()` calls with your own captured CSV and the number becomes yours.

19.8 Phase 4 — inference on the board

`train.py` writes `model.h`. Put it in `main/` next to `main.c`. This is the whole model — nothing is undefined.

```
// model.h -- GENERATED by train.py. Do not edit by hand.
#pragma once
#define W_RMS  +3.153609f
#define W_PEAK +8.625372f
#define BIAS   -2.058178f
#define GOLDEN_COUNT 3
#define GOLDEN_TABLE { \
    { 0.040f, 0.120f, 0.2897f }, \
    { 0.100f, 0.300f, 0.6995f }, \
    { 0.070f, 0.200f, 0.4719f }, \
}
```

Add these to `main.c`, above `app_main`:

```
#include "model.h"

// Decision threshold. NOT a round number picked by feel -- see 19.10.1, which
// measures the trade. At 0.80 this model misses 83% of faults.
#define THRESHOLD 0.45f

// FEATURE ORDER MUST MATCH TRAINING: index 0 = rms, index 1 = peak.
// Swap them and the model still runs, still returns a plausible 0..1 score,
// and is systematically wrong. Nothing crashes and nothing warns you.
enum { F_RMS = 0, F_PEAK = 1, F_COUNT = 2 };

static float model_predict(const float *f, int n)
{
    if (n != F_COUNT) return -1.0f; // contract check
    float z = W_RMS * f[F_RMS] + W_PEAK * f[F_PEAK] + BIAS;
    return 1.0f / (1.0f + expf(-z)); // logistic / sigmoid
}

// Golden vectors computed on the PC with the SAME weights. If the board does
// not reproduce these, the port is wrong -- catch it before trusting any live
// reading. This is the single most valuable thing in the lab.
static bool model_self_test(void)
{
    static const struct { float rms, peak, expect; } v[GOLDEN_COUNT] = GOLDEN_TABLE;
    bool all_ok = true;
    for (int i = 0; i < GOLDEN_COUNT; i++) {
        float f[F_COUNT] = { v[i].rms, v[i].peak };
        float got = model_predict(f, F_COUNT);
        bool ok = fabsf(got - v[i].expect) < 0.001f;
        if (!ok) all_ok = false;
        ESP_LOGW("model", "vector %d: got %.4f expect %.4f %s",
            i, (double)got, (double)v[i].expect, ok ? "PASS" : "FAIL");
    }
    return all_ok;
}
```

Then replace the CSV `printf` inside the window block with the decision, timed:

```

float rms, peak;
compute_features(win, WIN, &rms, &peak);

float f[F_COUNT] = { rms, peak };
int64_t t0 = esp_timer_get_time();
float score = model_predict(f, F_COUNT);
int64_t us = esp_timer_get_time() - t0;

if (score > THRESHOLD)
    ESP_LOGW("ml", "FAULT score %.3f rms %.4f (%lld us)",
              (double)score, (double)rms, us);
else
    ESP_LOGI("ml", "ok score %.3f rms %.4f (%lld us)",
              (double)score, (double)rms, us);

```

And call the self-test once from `app_main`, after `imu_init()`:

```

if (!model_self_test()) {
    ESP_LOGE("ml", "model self-test FAILED -- refusing to run inference");
    abort(); // a wrong port is worse than no port
}

```

19.9 Phase 5 — prove it

#	Do this	Expected
1	Boot and read the self-test.	All three vectors PASS . If any fails, your weights or feature order are wrong — fix that before anything else.
2	Run the clean fan for 60 s.	Mostly <code>ok</code> , scores below 0.45.
3	Add the coin, run 60 s.	Mostly <code>FAULT</code> , scores above 0.45.
4	[MEASURE] Read the microsecond figure in the log.	Record it. Compare against the 256 ms window — the ratio is your real-time headroom. Do not quote a number you did not read.
5	[MEASURE] Count misclassifications across steps 2 and 3.	That is your accuracy, on your rig. Report it as a number, not as "it worked."

19.10 Extensions, worked

NOTE

How numbers here are labelled

[SIMULATED] produced by `train.py` on synthetic data — reproducible, but not from your bench.

[DERIVED] arithmetic, shown. **[MEASURE]** you must run it. Only **[MEASURE]** results are yours to claim.

19.10.1 — Choosing the threshold, with the measured curve

An earlier edition said "set the threshold at 0.8 because a missed fault costs more than a false alarm." **The reasoning is right and the number is backwards**, and the sweep shows why. `train.py` prints this table.

KEY POINT

Held-out sessions[SIMULATED]****

Threshold	Missed faults	False alarms
0.30	0.0%	53.0%
0.40	3.0%	8.0%
0.50	9.0%	0.0%
0.60	27.0%	0.0%
0.80	83.0%	0.0%

At 0.80 the model **misses five faults in six**. A threshold chosen to sound "conservative" produced a detector that barely detects. If missed faults are what you fear you want a **low** threshold, not a high one — a high threshold means "only report when very confident," which is exactly how you miss things.

0.45 is the value in the listing: between the 0.40 and 0.50 rows, accepting a few percent of false alarms to keep missed faults low. Justify yours the same way, from your own curve.

"**I chose 0.45 because at 0.8 the model missed 83% of faults on held-out sessions**" is a complete engineering answer. It cites a measurement, names the trade, and shows you tested rather than assumed.

19.10.2 — What the model costs

[DERIVED] Three `float` constants is **12 bytes of learned parameters**. Be precise about that phrasing — the *model* costs 12 bytes, but the *feature* is not free: `model_predict` compiles to a few dozen instructions, and `expf` pulls in `libm`, which is the largest single contribution. Measure the real delta rather than quoting 12 bytes as the whole cost:

```
idf.py size                # build with inference, note .text and .rodata
# then comment out the model_predict call and the include, rebuild:
idf.py size                # the difference is the true cost
```

[MEASURE] Record both figures. On this class of part expect the honest answer to be a few hundred bytes to a couple of kilobytes, dominated by `expf` — not 12.

KEY POINT**Why this still beats reaching for TensorFlow Lite Micro here**

	This model	TFLite Micro, small NN
Learned parameters	12 bytes	a few KB
Interpreter / runtime	none — just libm	tens of KB of flash
RAM (tensor arena)	0	typically 2-20 KB
Explainable?	Yes — read the two weights	Not really

Being able to justify the simpler model is worth more in an interview than having used the framework. Reach for TFLite Micro when you genuinely need a nonlinear boundary over many features — raw audio, images, multi-axis spectra. Not for two hand-engineered features.

19.10.3 — Proving the feature-order bug is real

Deliberately swap the order and watch it fail quietly:

```
float f[F_COUNT] = { peak, rms }; // WRONG ORDER, on purpose
```

The self-test catches it immediately — that is what golden vectors are *for*, and why `app_main` aborts on failure. Without it you would see plausible scores, degraded accuracy, and no error at all, and you would probably blame your data.

Restore the correct order afterwards, then write the story down: **"I caught a feature-order bug with golden vectors before it reached live data"** is exactly the kind of specific, verifiable thing interviewers remember.

19.11 Being accurate about what this lab gives you

WARNING

Say this precisely, because an interviewer will probe it

You now have: **labelled data captured on real hardware, a trained model, exported weights, on-device inference verified against golden vectors, and measured accuracy, latency and memory cost.** That is a complete, if small, ML deployment pipeline, and most candidates who mention TinyML have not done it end to end.

You do **not** have: deep learning, a large dataset, a nonlinear model, or experience with a production ML toolchain. Do not imply otherwise.

If you captured `data.csv` and retrained, this is true and it is the strong version:

"I captured labelled vibration data on the board, trained a classifier off-device on held-out capture sessions, exported the weights, and verified the on-device port against golden vectors. Inference measured about N microseconds against a 256 millisecond window."

If you ran the lab with the supplied weights and did not capture your own data, that sentence is false and an interviewer will find out in one follow-up. Say this instead:

"I built the capture and inference pipeline end to end and validated the on-device port against golden vectors. The model I deployed was trained on synthetic data supplied with the exercise; I have not yet retrained it on my own captures."

The second sentence costs you nothing. It is still a complete embedded ML pipeline, and it is **recoverable** — the first one is not, if it turns out you never captured anything. Fill in the microsecond figure only if you read it from your own log.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

Additional corrections

Wrong declarator, and the earlier explanation of it was wrong too. `static volatile const char *g_label` declares a pointer to *volatile const char* — it makes the characters volatile, not the pointer. Since it is the *pointer* that the ISR reassigns, the correct form is `static const char * volatile g_label`. Better still, do not rely on `volatile` for cross-context handoff at all; post the label change through a queue or a task notification.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 19.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19 ←	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.